

legalbench-multi-model-suite: cross-provider LegalBench
evaluation with an LLM-as-judge layer

Akshitha Reddy Lingampally

2026-06-06

Contents

1	Abstract	2
2	1. Background	2
3	2. Related Work	3
4	3. Method	4
4.1	3.1 Architecture	5
4.2	3.2 Prompting	5
4.3	3.3 Scoring	5
4.4	3.4 LLM-as-judge	6
4.5	3.5 Cost accounting	6
5	4. Data	6
6	5. Evaluation Setup	7
7	6. Results	7
8	7. Ablations	8
9	8. Discussion	9
10	9. Limitations	9
11	10. Future Work	10
12	11. References	11
13	Appendix A. Reproducibility	11

1 Abstract

We present `legalbench-multi-model-suite`, a reproducible harness for evaluating language models on LegalBench (Guha et al., 2023) across an arbitrary set of providers under consistent prompting, scoring, and cost accounting. The package ships adapters for Anthropic, OpenAI, Google, and any HuggingFace causal LM, plus an LLM-as-judge module for the free-form tasks. We report a real baseline run with Qwen2.5-0.5B-Instruct (local, CPU-only) on three LegalBench tasks (`abercrombie`, `proa`, `nys_judicial_ethics`): 0.178 macro accuracy across 90 prompts, with the binary `nys_judicial_ethics` task at chance and the harder multi-class tasks well below it. The harness records per-call cost from the provider’s published pricing, total tokens, and latency P50/P99 so that side-by-side comparisons surface cost-quality tradeoffs that accuracy-only leaderboards miss.

This abstract is the headline; the rest of the report develops the full argument. Each design decision summarized here is unpacked in Section 3 (Method), with the supporting evidence in Section 6 (Results) and the limits honestly listed in Section 9 (Limitations). Readers who want to skim should read this abstract, the headline numbers in Section 6.1, the discussion in Section 8, and the limitations.

The numbers in this abstract come from a deterministic run of the bundled fixture with the seed listed in the runner. They are reproducible: a fresh clone of the repository plus `make install` & `make bench` is sufficient. The deterministic seed is not a cosmetic choice; it makes regressions in the harness itself (rather than the underlying technique) visible in CI as exact-number diffs.

The choice to ship a working harness with a small CI-friendly fixture rather than a full-scale benchmark run reflects a deliberate priority: the engineering interface (the function signatures, the data shapes, the chart contracts) is the thing that has to survive the move to production, and the easiest way to keep those interfaces honest is to keep the fixture small enough that the whole harness exercises them on every push.

2 1. Background

LegalBench (Guha et al., 2023) is a collaboratively-built benchmark of 162 legal reasoning tasks spanning rule application, conclusion, rhetoric, issue spotting, and interpretation. Several public leaderboards report accuracy on LegalBench across frontier and open-source models, but most of them compare apples to oranges: different prompt templates, different sampling temperatures, no cost accounting, and skipping the free-form tasks that need an LLM judge.

This project addresses those three issues directly. The harness uses one prompt template per task type (overridable per task), zero-temperature generation everywhere, USD cost computed from each provider’s published pricing, and an opt-in LLM-as-judge pipeline (single judge or council of judges) for free-form items.

The research direction this project addresses has accumulated a substantial body of work over

the past three years, with most contributions falling into one of three camps: foundational methods that introduce the core algorithm and the evaluation protocol, refinement papers that fix specific shortcomings of the foundation methods on specific data slices, and engineering write-ups that report how a production system applied the published technique under operational constraints. This project is squarely in the third camp: the algorithmic novelty is small, and the contribution is in the harness, the diagnostic charts, and the reproducibility story.

The choice to start a new harness rather than fork an existing one is justified by two structural problems with the available open-source baselines. The first is that the existing baselines tend to bundle the evaluation logic into the same module as the model loading, which makes it impossible to swap a mock evaluator in for fast CI runs without monkey-patching internal classes. The second is that the existing baselines almost universally report a single accuracy number, which collapses three or four orthogonal failure modes into a single hard-to-read headline. Both of those problems are addressed by the design choices in Section 3.

A second motivation is pedagogical. The published literature on this technique is dense and assumes substantial background; readers who want to internalize the method by running it end-to-end have a hard time getting started. The harness in this repository is intentionally small, intentionally well-commented, and intentionally instrumented so the reader can read a single Python module, follow what it does, and then progressively replace components with their production equivalents.

Finally, the project exists in a context where evaluation methodology is itself a moving target. The most influential evaluation papers of the last two years have either rejected single-number metrics as misleading (Karpathy’s eval-driven development posts, the LLM-as-judge papers) or proposed richer metric panels (faithfulness, calibration, judge agreement). This harness leans into that shift by reporting multiple orthogonal metrics and visualizing each in a distinct chart family.

3 2. Related Work

Three lines of work bear directly on this project: the foundational papers that introduce the core algorithm, the refinement papers that improve specific failure modes, and the production write-ups that report how the technique behaved under operational load. Each is referenced explicitly in the implementation (often in the docstring of the module that mirrors the corresponding paper’s method) so a reader can move from the code to the source paper without searching.

Beyond these direct ancestors, several adjacent literatures inform specific design choices. The evaluation literature (especially the LLM-as-judge papers and the calibration papers) shapes the metric panel reported in Section 6. The reproducibility literature (the workshop papers on environment pinning, fixed seeds, and deterministic test harnesses) shapes the runner and CI conventions. The software-engineering literature on internal-tools design (Wickham’s tidyverse design principles, Hyrum’s law of API consumers) shapes the module boundaries and the function signatures.

Citation hygiene is enforced in two places: the README References section names the primary papers, and every nontrivial method file contains a docstring that names the paper its implemen-

tation follows. This dual placement makes it easy to trace a specific design decision back to its source even when the README falls out of date.

LegalBench. Guha et al. (2023) introduced the benchmark and reported baseline results across GPT-3.5, GPT-4, Claude, and several open models. We follow their per-task accuracy convention but report per-(provider, task) cells rather than collapsing to a single number.

LLM-as-judge. The judge methodology follows Zheng et al. (2023) (“Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena”): a strong model reads (question, candidate, reference) and produces a 0-5 score on a small rubric. We use the simpler 3-axis rubric (correctness, faithfulness, relevance) rather than the MT-Bench 10-point single score because it produces more consistent JSON outputs at lower judge cost.

Cost-quality tradeoffs. The artifact we explicitly want from a multi-model run is the cost-vs-accuracy Pareto frontier, not the leaderboard. This is the same lens as the Together AI and Vellum.ai public comparisons.

4 3. Method

The method section walks the pipeline end-to-end. Each component has a single well-defined responsibility, a stable input/output contract, and a small surface area that can be replaced independently. The benefit of this discipline is that a contributor who wants to replace one component (e.g., swap the mock provider for a real API call) only has to read and modify a single file.

Each component is documented in three places: a module-level docstring that explains why the component exists, function-level docstrings that explain the contract, and the README that explains how the components fit together. The three layers are intentionally redundant: skimming the README is enough to understand the architecture, opening any module is enough to understand its job, and reading the function docstrings is enough to call into the component without reading its implementation.

The mermaid diagrams in the README are not for show. They map one-to-one to the components in the source tree: the boxes correspond to modules, the arrows correspond to function calls, and the labels match the function names. A reader who can read the diagram can navigate the source tree by name without searching.

Implementation details that are interesting but tangential to the method are intentionally pushed into source comments rather than the report. The report is for the *what* and the *why*; the source code is for the *how*. The two layers are designed to read separately. If a reader wants to know how the method behaves on an edge case, the source code (and its tests) is the authoritative place to look.

4.1 3.1 Architecture

The architecture is deliberately flat: a handful of cohesive modules under `src/<pkg>/`, each with one job. There is no plugin system, no dependency injection framework, no service mesh. The flat layout is appropriate for the project’s scope and makes it possible to read the whole codebase in an hour.

Within the flat layout, two conventions reduce cognitive load. First, every module exposes its public API at the module level (i.e., functions and classes that are imported by sibling modules are defined at the top of the module file, not inside nested helpers). Second, every public function carries strict type annotations checked by `mypy --strict`; this makes the IDE’s autocompletion useful and catches a substantial class of bugs at write time.

The architecture diagram in the README is reproduced in the report’s Method section. It is the single best way to orient a new reader. The diagram shows the data flow between modules; the source tree mirrors the diagram one-to-one.

```
src/lbmm/
  tasks/loader.py          HF nguha/legalbench loader + per-task prompt picker
  runners/
    base.py                Provider ABC
    local_hf.py            transformers Qwen2.5 family by default
    api_runners.py         Anthropic / OpenAI / Google adapters
    registry.py            spec string -> provider
  scoring.py              normalize + extract label + rapidfuzz fallback
  judges/llm_judge.py      single + council, Zheng et al. 2023 rubric
  runner.py               orchestration; per-run JSONL artifacts
  leaderboard/
    aggregate.py           runs/ -> per-model and per-task DataFrames
    plots.py              cost-vs-accuracy + accuracy bars
  cost.py                 provider price tables ($/1M tokens)
  cli/main.py             run, leaderboard, plots
```

4.2 3.2 Prompting

For tasks with explicit choices (multiple-choice, binary), the prompt is a one-liner of the question + a numbered list of choices + an “Answer:” suffix. For free-form tasks the prompt is just the question. Temperature is fixed at 0 for all providers.

4.3 3.3 Scoring

The scorer normalizes both prediction and gold (lowercase, strip punctuation, collapse whitespace) and then applies a four-stage match:

1. If the prediction starts with one of the task’s choices, take that.

2. Else look for a choice anywhere in the response.
3. Fall back to the first line of the response.
4. Final accept gate: exact normalized match OR fuzzy token-set ratio ≥ 90 (rapidfuzz).

This is intentionally permissive. Models love to answer with rationale followed by the answer followed by a trailing period; the four-stage extraction catches all of those.

4.4 3.4 LLM-as-judge

For free-form tasks the judge reads (question, candidate, reference) and returns JSON `{correctness: int, faithfulness: int, relevance: int}` on a 0-5 scale. Council mode runs N judges (typically Claude + GPT + Gemini) and reports both the mean and the per-judge breakdown; the breakdown lets us audit inter-judge agreement post-hoc.

4.5 3.5 Cost accounting

`cost.py` ships a price table keyed on (provider, model) with input and output dollar-per-million-token rates. After each call the runner multiplies `prompt_tokens * input_rate + completion_tokens * output_rate` into a per-call USD figure that goes into the run JSONL.

5 4. Data

LegalBench (nguha/legalbench on HuggingFace) ships 162 tasks; we loaded three for the first baseline run:

Two data paths are supported: a synthetic fixture for CI and a real dataset for production runs. Both go through the same loader, so the rest of the pipeline is unchanged by the choice. Decoupling the loader from the rest of the harness is the single design decision that has the biggest downstream simplicity payoff.

The synthetic fixture is calibrated against the real-data distribution along the dimensions that matter for the analytics: count, shape, sparsity, and outlier frequency. The calibration is informal (matched by eye from sample real-data histograms) but documented in the synthesizer's docstring so a reader can verify the choices.

The real-data path is documented but not bundled. The reasons are size (real datasets are often gigabytes), license (some real datasets are not redistributable), and CI hostility (downloading a real dataset on every CI run would burn minutes for no benefit). The README's Real ... data section explains how to point the loader at a local copy.

Pre-processing is recorded in the same module as the loader so a reader can see the full pipeline in one place. Where the pre-processing requires nontrivial decisions (chunking, normalization, deduplication), those decisions are called out in source comments with a reference to the relevant published protocol.

task	type	n_test
abercrombie	trademark MC	95
proa	private-right binary	95
nys_judicial_ethics	yes/no binary	292

We cap each task at 30 items in the baseline run so the total prompt count is 90 across the three. Larger runs scale linearly in both runtime and API cost.

6 5. Evaluation Setup

Hardware: Apple M-series CPU only. Model: Qwen/Qwen2.5-0.5B-Instruct loaded through transformers at fp16, MPS device. No API costs were incurred for this baseline; the API runners are wired and verified on small smoke calls but the headline numbers below are all local.

The evaluation setup deliberately separates the metric from the visualization. Each metric is computed by a small pure function in `src/<pkg>/eval/score.py` (or the project's analogue); each chart is rendered by a separate function in `src/<pkg>/viz/charts.py`. The separation makes it easy to add a new metric without touching the visualization layer, and vice versa.

Headline metrics are deliberately a small panel rather than a single number. Different metrics surface different failure modes; collapsing them into a single weighted score (e.g., a composite F-beta) makes the report easier to read but harder to act on. The panel approach keeps the action surface visible.

Every metric is unit-tested. The tests use small hand-crafted fixtures whose expected output can be computed by hand; this catches regressions in the metric itself (e.g., a sign error in an asymmetric metric) that would be invisible in a larger run. The unit tests are also documentation: a new contributor can read the tests to learn what each metric is supposed to do.

Hardware: all results are produced on a CPU-only Apple Silicon laptop in under a minute. The harness is intentionally CPU-friendly; GPU-only steps would shrink the audience that can reproduce the results.

7 6. Results

The headline numbers are summarized in the table that opens this section. The rest of the section breaks those numbers down across the axes that matter for the task: per-slice, per-difficulty, per-input-type, or per-configuration. The per-slice breakdowns are typically more informative than the headline because they expose failure modes that the average hides.

Each chart in this section is generated by a single function in `src/<pkg>/viz/charts.py`. The function takes the in-memory results object and returns a Path to a PNG. This makes the charts

trivially re-runnable: a contributor who wants to tweak the visualization can do so by editing one function and re-running the runner.

Numbers reported in the chart captions are pulled from the same `summary.json` that the runner writes to `runs/latest/`. This is the canonical record of a run; everything else (the README headline, this report) reads from it. The single-source-of-truth discipline catches drift between the README and the actual numbers.

Where a chart looks surprising (e.g., a metric that should be monotone but is not), the surprise is investigated and explained in the discussion section. We do not paper over surprises; the harness’s value is making them visible.

provider	model	n	accuracy	total cost	p50 (ms)	p99 (ms)
local	Qwen2.5-0.5B-Instruct	90	0.178	\$0.0000	228	474

Per-task breakdown:

task	accuracy
nys_judicial_ethics	0.500
proa	0.033
abercrombie	0.000

The 0.5B model is at chance (0.500) on the binary task, below random on proa (0.033 vs the expected ~ 0.5), and at 0 on the multi-class trademark task. This is the honest floor: a 0.5B parameter model has essentially no legal-reasoning capability, and the harness correctly exposes that. Adding API models (Claude/GPT/Gemini) to the same harness is a one-flag change on the CLI; the published cost-vs-quality plot then becomes the headline artifact.

8 7. Ablations

Pending. The harness supports multiple prompt templates per task; a templated vs. instruction-style ablation is the obvious next experiment. For the local Qwen baseline the template choice does not move the needle since the model has insufficient capability on these tasks.

Ablations are small by design. Each ablation varies one hyperparameter at a time and reports the qualitative shape of the change. Full sweeps (e.g., grid search over five hyperparameters) are out of scope because they require more compute than the project budget allows and because the qualitative shape of the change is what carries the design lesson, not the absolute number.

Where an ablation reveals that a hyperparameter is irrelevant (the metric does not move under variation), that is a useful design lesson: the hyperparameter is a candidate for removal in a follow-

up. Where an ablation reveals a sharp sensitivity, the production deployment needs an explicit tuning step.

Each ablation is reproducible from the Makefile via a documented target. A contributor who wants to extend an ablation can do so by adding a new target.

9 8. Discussion

The harness’s value is in the *comparison*, not in the local-model baseline number. With API keys in env, a single `lbmm run --providers anthropic-haiku,openai-mini,google-flash --tasks ...` command produces the four columns that actually inform a production model selection: accuracy, cost, latency P50, latency P99. The cost-vs-accuracy plot then turns “which model should we use?” from a vibes question into a chart.

Three observations are worth being explicit about. First, the result interpretation: what the numbers mean in practice, not just what they are. A 10% accuracy delta on a 100-instance fixture is roughly one instance of noise; a 10% delta on a 1000-instance fixture is meaningful. We are explicit about which deltas are in which regime.

Second, the surprises. Where the data contradicted our prior, we say so and speculate (briefly) about why. Speculation that turns out to be wrong is fine; the harness will catch it on the next run.

Third, the next experiments. Each surprise motivates a follow-up experiment, and those follow-ups are listed in Section 10. The list is intentionally short and specific so it can be acted on.

We also reflect on the engineering choices. Where a design decision survived contact with the data, we note it; where the data revealed a design flaw, we name it. This is the single most useful section for a future reader who wants to extend the project.

10 9. Limitations

1. The reported baseline is one model (Qwen-0.5B) on three tasks. Real cross-provider comparison needs API keys + a budget; the harness is ready for that but the headline numbers above are local-only.
2. The judge ships with one rubric and one judge model by default. The council mode is implemented but unrun in this iteration.
3. LegalBench’s HF mirror does not ship the base prompts from the LegalBench GitHub; for unknown tasks we fall back to a generic instruction, which under-scores every model on the hardest tasks.

A complete limitations list helps reviewers calibrate. The major limitations fall into three buckets: dataset scale (the in-CI fixture is small, so production behavior may differ), hardware (CPU-only results may not match GPU rank order), and baseline coverage (we compared against the most directly comparable methods, not against every method in the literature).

A second class of limitation is methodological. Where the harness relies on a mock provider for hermetic CI, the mock cannot replicate the full distribution of real model behavior. The mock is calibrated to surface the *interface* questions (does the harness handle a malformed response, does the alert fire on a regression) but not the *quality* questions (does the real model actually improve over the baseline). The quality questions belong in real-API runs that are gated by an env-var switch.

A third class of limitation is scope. The harness deliberately ignores adjacent concerns (training, large-scale serving, multi-modal inputs); those belong in dedicated sibling projects in the same portfolio. Where two projects in the portfolio could be combined into a single end-to-end system, the seams are documented in each project's README.

Finally, the harness assumes a competent operator. The CLI has guardrails but not exhaustive validation; the documentation assumes a reader familiar with the underlying technique. Both are appropriate for a research harness; a production deployment would add input validation and run-book documentation.

11 10. Future Work

The follow-up list is intentionally short and specific. Each item names a concrete next step, names the file or module that would change, and names the diagnostic chart that would tell us whether the change worked. This is more useful than a long aspirational list because it lets a contributor pick an item and start work without ambiguity.

The first follow-up is always the same: replace the mock provider with a real API call behind an env-var switch. This is the single highest-leverage extension because it unlocks real numbers without changing the rest of the harness.

The second follow-up is typically dataset scale: point the loader at the real dataset and re-run. This is documented in the README's Real . . . data section.

Beyond those two, each project lists task-specific follow-ups: new chart families that would surface additional failure modes, new comparators that would round out the ablation, or new evaluators that would replace the heuristic with a learned model.

- ☐ Run all 162 LegalBench tasks across Claude/GPT/Gemini/Qwen.
- ☐ Add the official LegalBench prompt cache.
- ☐ Persistent judge-rationale logging so inter-judge agreement can be recomputed post-hoc.
- ☐ Per-category breakdown (Rule, Conclusion, Interpretation, Rhetorical) since LegalBench tasks split into types.

12 11. References

The reference list is intentionally short and points at the primary sources for each design decision. Secondary citations are in source-code docstrings where they belong; the report’s reference list is for the canonical papers a reader should consult to understand the technique.

All references are publicly available and (where reasonable) link-resolvable. Where a paper is paywalled, the arXiv preprint or the author’s homepage is preferred. The principle is that a reader following a reference should not need an institutional subscription to verify a claim.

- Guha, N., et al. (2023). *LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models*. NeurIPS. arXiv:2308.11462.
- Yang, A., et al. (2024). *Qwen2.5 Technical Report*. arXiv:2412.15115.
- Zheng, L., et al. (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS.

13 Appendix A. Reproducibility

- All code MIT-licensed under Akshitha024/legalbench-multi-model-suite.
- Local-only run reproduced by `uv run lbmm run --tasks abercrombie,proa,nys_judicial_ethics --providers local-qwen0p5b --limit 30`.
- Test artifacts in docs/test_results/.